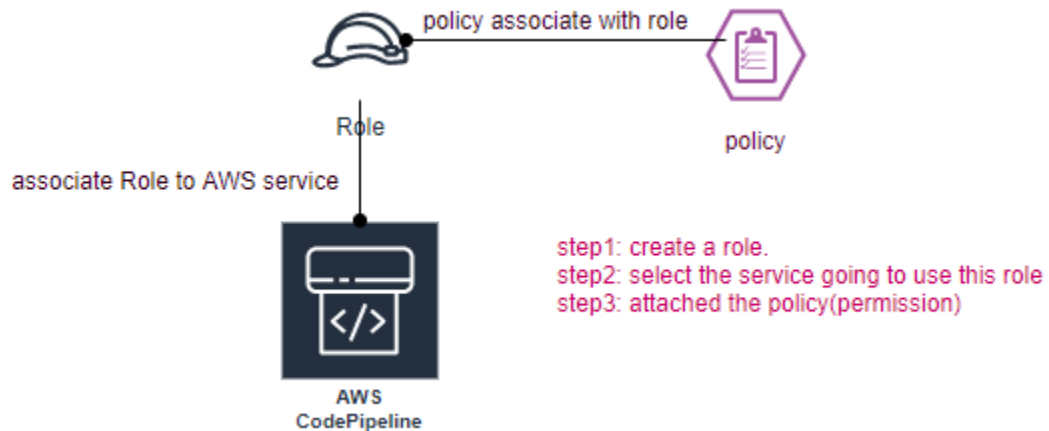


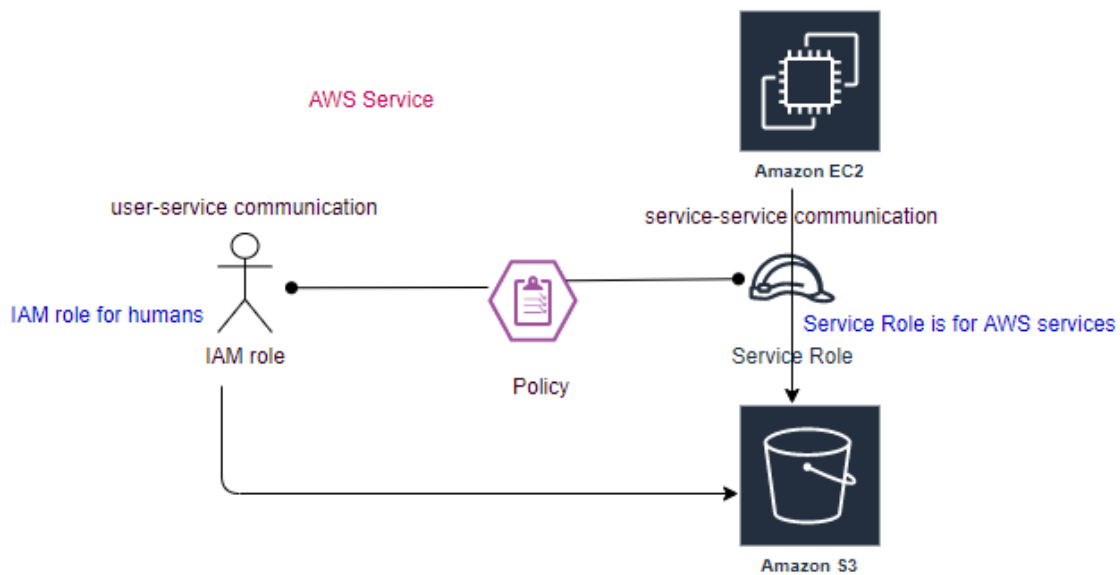
Day5:

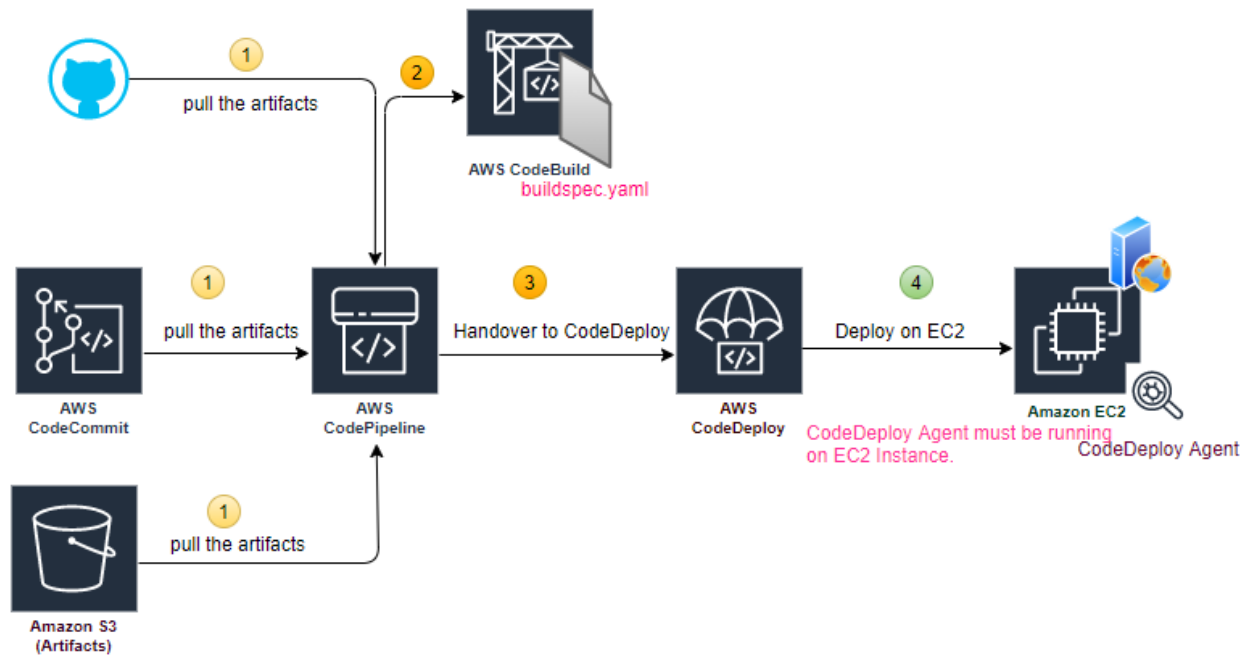
Service Role for CodeDeploy?

Service-to-Service Communication through roles, and role carry the necessary permissions.

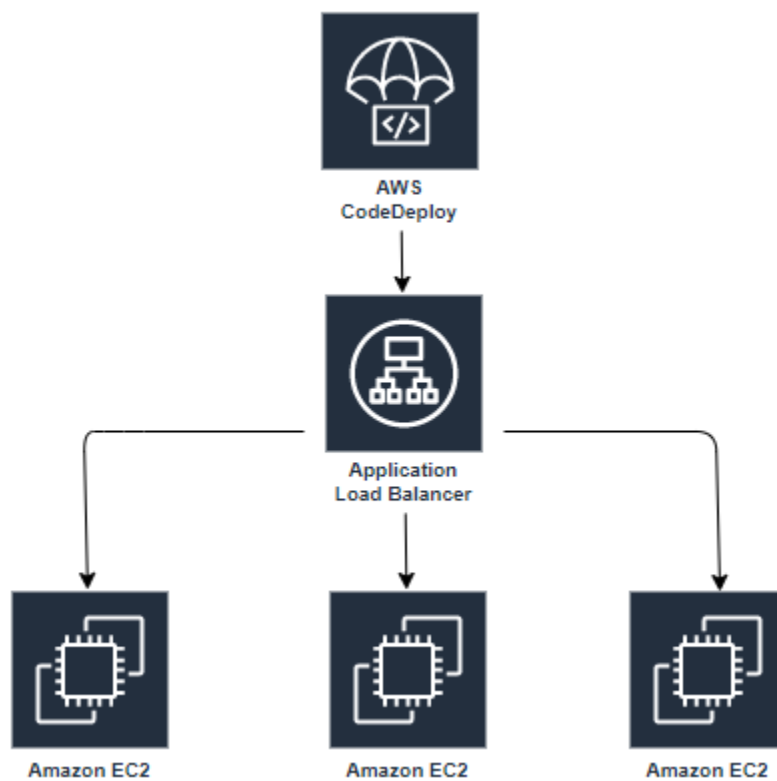


Difference between IAM role /Service role?

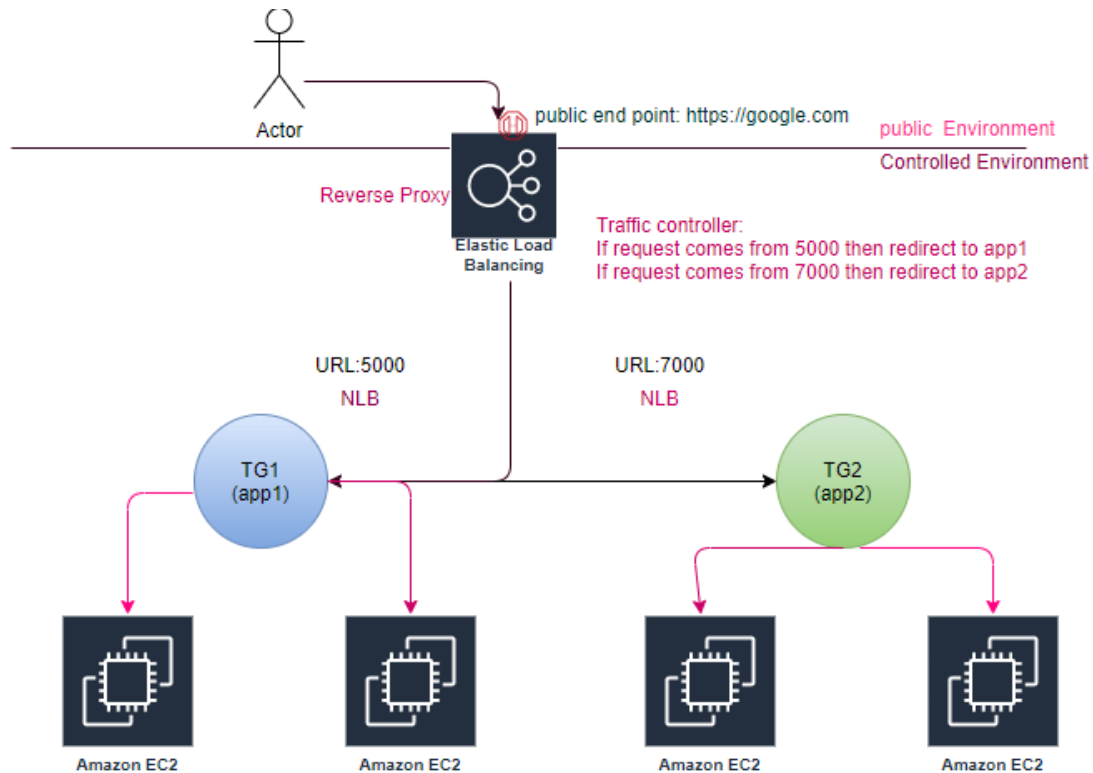




In real world there are multiple EC2 instances, Load is balanced by ELB. App is on top of EC2.

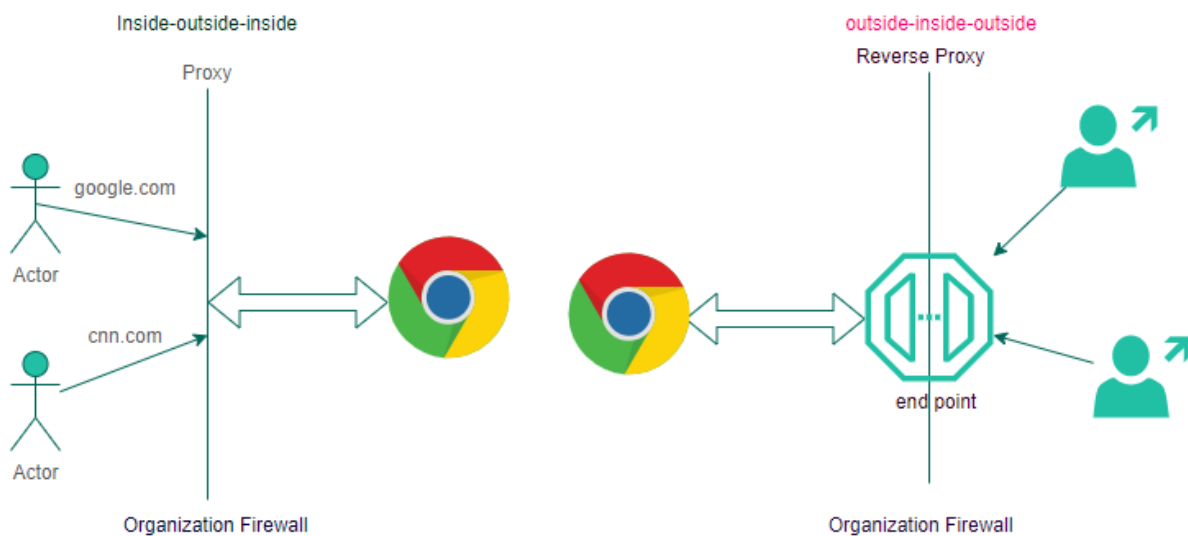


UC: LB

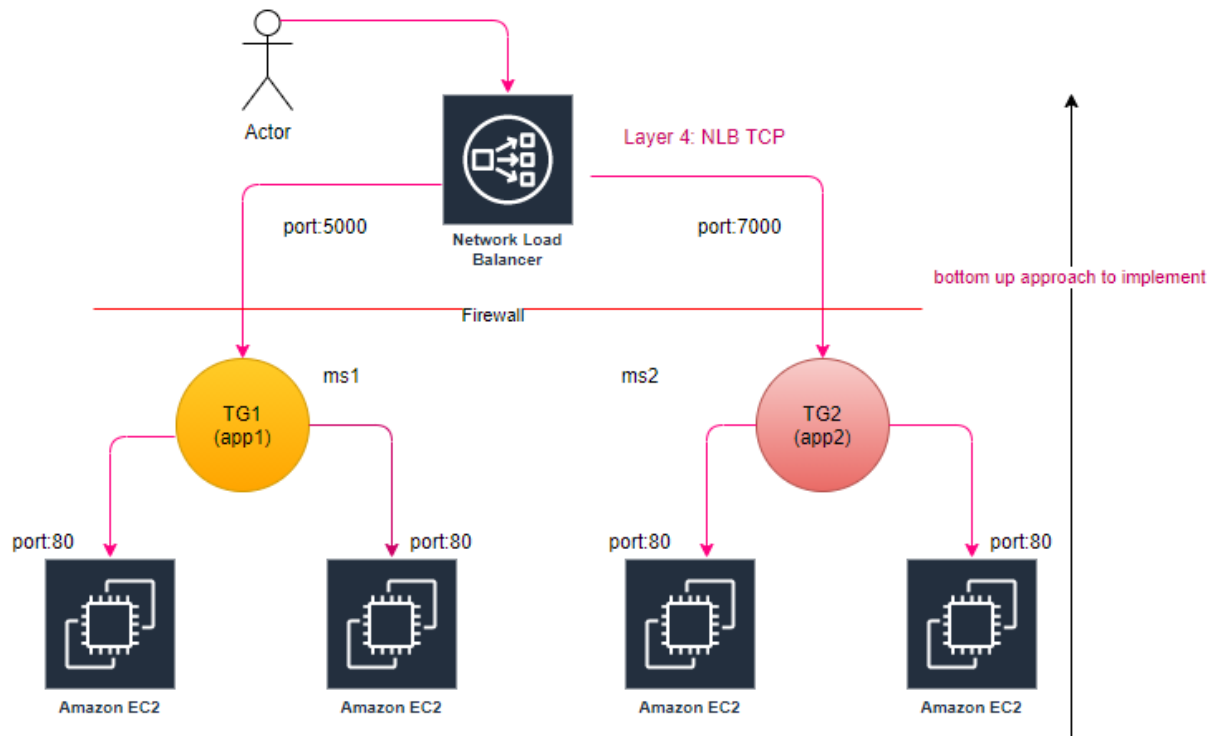


Endpoint service is just like a traffic controller (API Gateway or Reverse Proxy).

What is difference between proxy and reverse proxy (end point for external world).



ELB (Elastic Load Balancing): NLB uses burst algo to balance the traffic.



step1: Launch 2-2 instances per AZ

step2: Create TG for NLB (TCP), actual load balancing happens at TG level, **health check always be HTTP**

Target groups (2) info							Create target group
Search or filter target groups							Actions
☐	Name	ARN	Port	Protocol	Target type	Load balancer	
☐	TG-App-01-5000	arn:aws:elasticloadbalancin...	80	TCP	Instance	NLB-01	
☐	TG-App-02-7000	arn:aws:elasticloadbalancin...	80	TCP	Instance	NLB-01	

step3: Register the Instaces (targets) with TG.

step4: Configure Routing → Configure Target Group

Mapping-> corresponding to each subnet(routing) a seprate NIC is created.

Listeners and routing [info](#)

A listener is a process that checks for connection requests, using the protocol and port you configure. Traffic received by the listener is then routed per your specification.

▶ Listener TCP:5000 Remove

▼ Listener TCP:7000 Remove

Protocol	Port	Default action	
TCP ▼	7000 1-65535	Forward to	TG-App-02-7000 Target type: Instance
			TCP ▼ ↻

[Create target group](#)

These ports are immediately dropped as traffic routed

Validate:

- Is NLB end point is active.
- Is target group's targets are healthy
- This is also called reverse proxy?

You can host multiple applications behind single end point and redirect the traffic based on the port number.

Tab: Integrated Services

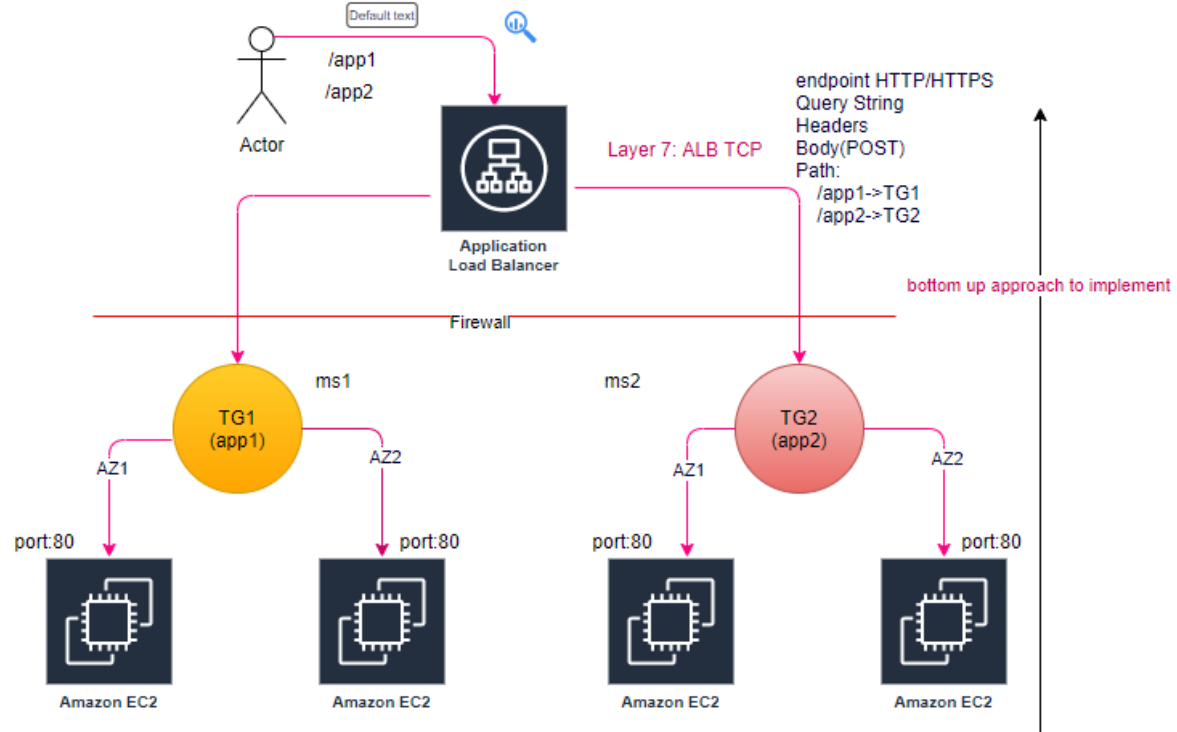
- VPC Endpoint Service (AWSPrivateLink) Layer4
 - You can create your NLB as a service, for others to consume as a VPC Endpoint Service.
- Traffic Mirroring (Layer4)
- Global Configuration (pop)
- Config (record each activity in the account)

Endpoint: DNS From consumer point of view

Endpoint Service-> publisher/provider

All applications worked at region labels.

ELB (Elastic Load Balancing): ALB, uses round robin algo to balance the traffic.



step1: Launch 2-2 instances per AZ

step2: Create TG for NLB (TCP), actual load balancing happens at TG level, **health check always be HTTP**

Target groups (1/3) info							
Search or filter target groups				Actions		Create target group	
	Name	ARN	Port	Protocol	Target type	Load balancer	
☐	TG-01	arn:aws:elasticloadbalancin...	80	HTTP	Instance	-	
☐	TG-App-01-5000	arn:aws:elasticloadbalancin...	80	TCP	Instance	NLB-01	
☒	TG-App-02-7000	arn:aws:elasticloadbalancin...	80	TCP	Instance	NLB-01	

step3: Register the targets with TG.

step4: Configure Routing → Configure Target Group

Mapping → corresponding to each subnet(routing) a seprate NIC is created.

Validate:

- Is ALB end point is active.

- Is target group's targets are healthy
- This is also called reverse proxy?

You can host multiple applications behind single end point and redirect the traffic based on the http/https, host-based routing (IP), path-based routing, http header-based, http-method based, Query String parameter-based, Source IP address CIDR based.

Tab: Integrated Services

- AWS WAF (Layer7)
- Global Configuration (pop)
- Config (record each activity in the account)

How to set Query based load balancing?

Tab: Listener

step1: Click on view/edit rule

ALB-01 | HTTP:80 (2 rules)

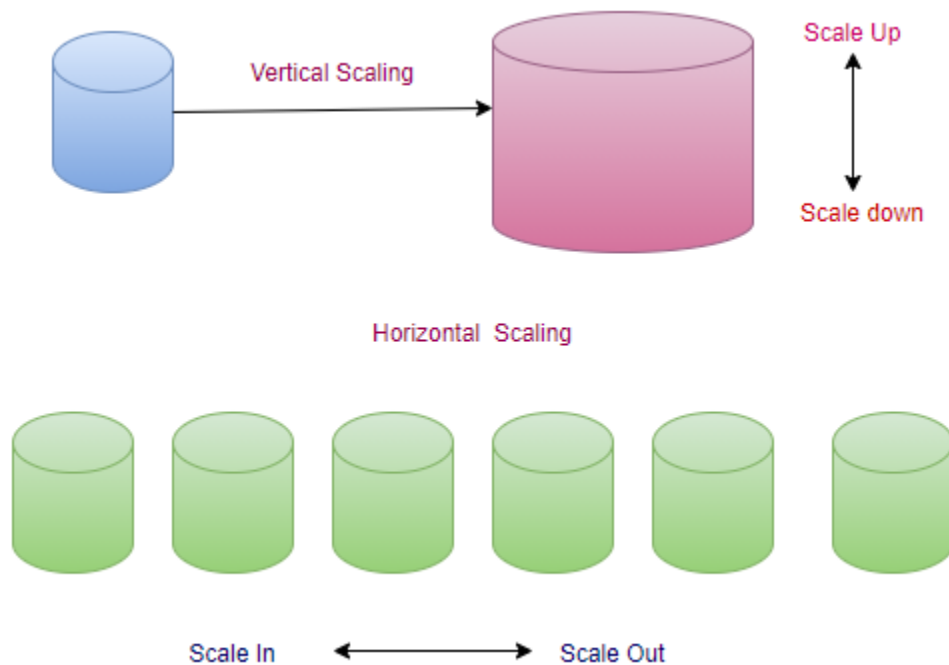
► Rule limits for condition values, wildcards, and total rules.

RULE ID	IF (all match)	THEN
1 A rule ID (ARN) is generated when you save your rule.	+ Add condition Host header... Path... Http header... Http request method... Query string... Source IP...	+ Add action THEN Forward to TG-01: 1 (100%) Group-level stickiness: Off
last HTTP 80: default action This rule cannot be moved or abandoned		

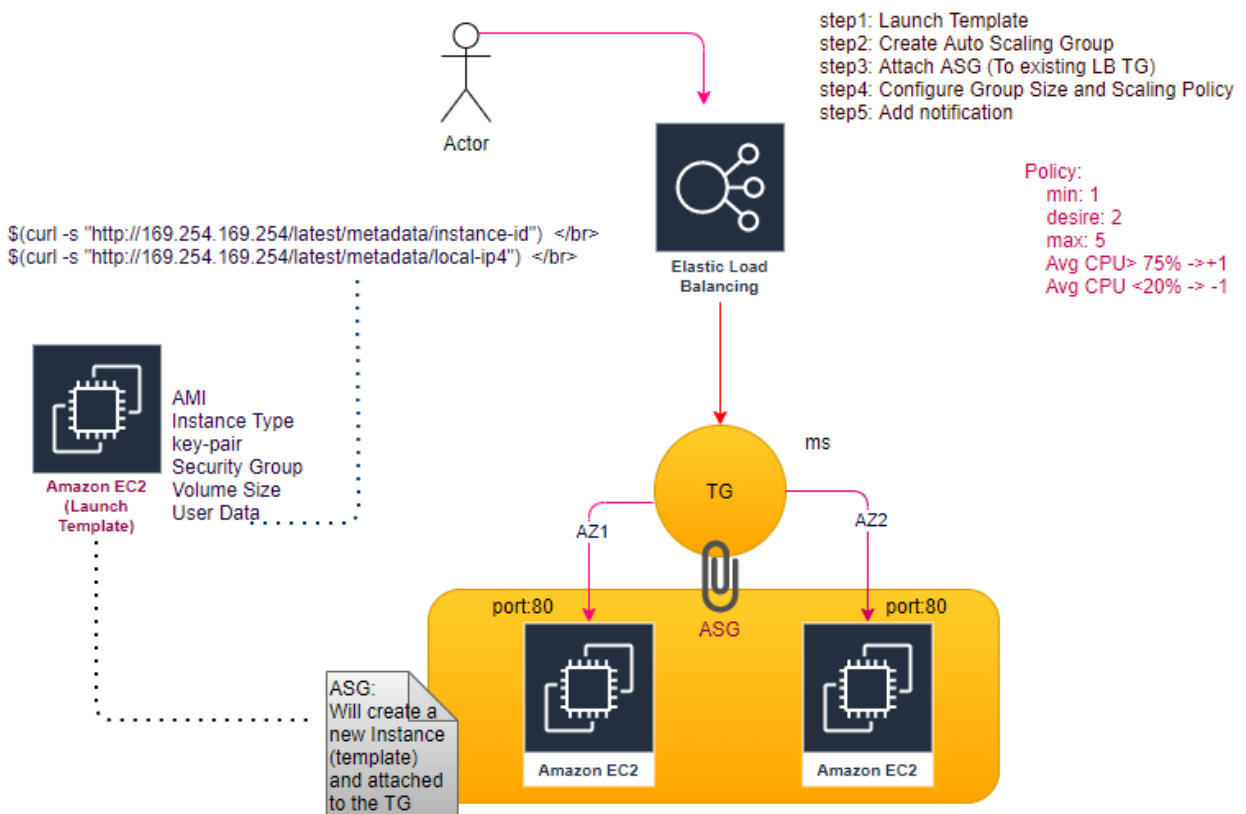
#	Routing Option	Route Client Request based on
1	Host-based	HOST field in HTTP header
2	Path-based	URL path on HTTP header
3	HTTP header-based	Value of any standard field or custom HTTP header
4	HTTP method-based	Any standard or custom HTTP method
5	Query String parameter-based	Query String or Query Parameter
6	Source IP address CIDR-based	Source IP address CIDR from where the request originates

All applications worked at region labels.

Autoscaling:



Autoscaling (HPA):



step: Just map Auto Scaling group with the target group, along with policy, will automatically create EC2 instance as per policy.

Attach to an existing load balancer
Select the load balancers that you want to attach to your Auto Scaling group.

☒ **Choose from your load balancer target groups**
This option allows you to attach Application, Network, or Gateway Load Balancers.

☐ **Choose from Classic Load Balancers**

Existing load balancer target groups
Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.

Select target groups ▼

⌂

TG-01 | HTTP
Application Load Balancer: ALB-01

✕

Scaling policies - *optional*

Choose whether to use a scaling policy to dynamically resize your Auto Scaling group to meet changes in demand. [Info](#)

☒ **Target tracking scaling policy**
Choose a desired outcome and leave it to the scaling policy to add and remove capacity as needed to achieve that outcome.

☐ **None**

Scaling policy name

Target Tracking Policy - scale up

Metric type

Average CPU utilization ▲

🔍 *Search metric types*

Average CPU utilization

Average network in (bytes)

Average network out (bytes)

Application Load Balancer request count per target

☐ Disable scale in to create only a scale-out policy

Vertical Scaling of EC2, first stop the Instance then change the Instance type.

UC: CodeDeploy (App01): more than one Instances (No LB)

step0: Artifacts available is S3 bucket.

step1: tag the both EC2 instances of app1

step2: CodeDeploy->Applications->Create Application

step3: Create Deployment Group (Group of more than 1 Instances)

step4: Environment configuration: Amazon EC2 Instances

☐ Amazon EC2 Auto Scaling groups

☒ Amazon EC2 instances
2 unique matched instances. [Click here for details](#)

You can add up to three groups of tags for EC2 instances to this deployment group.
One tag group: Any instance identified by the tag group will be deployed to.
Multiple tag groups: Only instances identified by all the tag groups will be deployed to.

Tag group 1

Key	Value - optional	
App01		Remove tag

☐ On-premises instances

Matching instances
2 unique matched instances. [Click here for details](#)

Will be failed, because CodeDeployment agent is not running on EC2 instances.

EC2 instance need also to contain IAM role to interact with the S3.

step5: CodeDeploy->Application-> Select the application->Deployment's tab-> Create Deployment.

Artifacts are in S3, as of now we were saying CodePipeline, now CodeDeploy itself will deploy using create deployment.

Application
CodeDeploy-App01

Deployment group

Q Deploy-App01-Revisions X

Compute platform
EC2/On-premises

Revision type

☒ My application is stored in Amazon S3 ☐ My application is stored in GitHub

Revision location
Copy and paste the Amazon S3 bucket where your revision is stored

Q s3://edureka-codedeploy-source-001/App-02.zip X

s3://bucket-name/folder/object.[zip|tar|tgz]

Revision file type

zip ▼

As of now we don't have assigned role to EC2 to communicate with S3, and also don't install CodeDeploy agent on EC2.

step6: Add the IAM role in both of the Instances (Security->Modify IAM role).

step7: Connect to the EC2 instance and install the CodeDeploy agent.

step8: Click on re-deployment

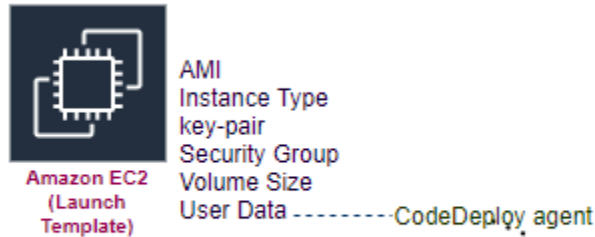
CodeDeploy and TargetGroup never talk to each other, Instances are the interface between CodeDeploy and TargetGroup. LB main objective is just traffic control.

We are just deploying new application only on 2 EC2 instances (app1).

EC2 can be connected with more than one target group.

Auto Scaling Group

Launch Template/Configuration



step1: Create Launch Template (IAM role CodeDeploy): Autoscaling group need to scale the instance. (Launch Configuration deprecated).

step2: Create Auto Scaling Group (min, desired, max instances)

step3: Associate Target Group with Auto Scaling Group (In case of LB association).

Load balancing - optional [Info](#)

Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.

☐ No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.

☒ Attach to an existing load balancer
Choose from your existing load balancers.

☐ Attach to a new load balancer
Quickly create a basic load balancer to attach to your Auto Scaling group.

Attach to an existing load balancer
Select the load balancers that you want to attach to your Auto Scaling group.

☒ Choose from your load balancer target groups
This option allows you to attach Application, Network, or Gateway Load Balancers.

☐ Choose from Classic Load Balancers

Existing load balancer target groups
Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.

Select target groups ▼ ↺

step4: Configure group size and scaling policies.

Configure group size and scaling policies [Info](#)

Set the desired, minimum, and maximum capacity of your Auto Scaling group. You can optionally add a scaling policy to dynamically scale the number of instances in the group.

Group size - *optional* [Info](#)

Specify the size of the Auto Scaling group by changing the desired capacity. You can also specify minimum and maximum capacity limits. Your desired capacity must be within the limit range.

Desired capacity

Minimum capacity

Maximum capacity

Autoscale Group Launches the EC2 automatically based on the policy defined (don't need to create EC2 instances manually).

UC: CodeDeploy (App01): Autoscale

step5: CodeDeploy-> Create application

step5.1: Create deployment group

step5.2: Environment configuration=> Amazon EC2 Auto Scaling Group

Environment configuration

✓ Amazon EC2 Auto Scaling groups
2 unique matched instances. [Click here for details](#)

Demo-ASG-001

☐ Amazon EC2 instances

Additional deployment behavior settings

ApplicationStop lifecycle event failure - *optional*
Type a deployment group name

☐ Don't fail the deployment to an instance if this lifecycle event on the instance fails

Content options - optional

☐ **Fail the deployment**
An error is reported and the deployment status is changed to Failed.

- **Overwrite the content**
The file in the application revision is copied to the target location on the instance, replacing the previous file.

- ☐ **Retain the content**
The file in the application revision is not copied to the instance. The existing file is kept at the target location and treated as part of the new deployment.

step7: CodeDeploy->Application-> Select the application->Deployment's tab-> Create Deployment

CodeDeploy itself will deploy using create deployment.

Autoscaling, request CodeDeploy and says hey! now we are associated to each other, i have launched a new instance, please deploy the newer version on new instance.

UC: Associate ASG with LB?

Edit the ASG:

Load balancing - optional

Load balancers

☒ Application, Network or Gateway Load Balancer target groups
Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.

Select target groups

☐ TG-01-5000 | TCP
Network Load Balancer: NLB-01

☐ TG-02-7000 | TCP
Network Load Balancer: NLB-01

☒ TG-App1 | HTTP
Application Load Balancer: ALB-01

☐ TG-App2 | HTTP
Application Load Balancer: ALB-01

step2: Validate from TG

TG-> Targets tab

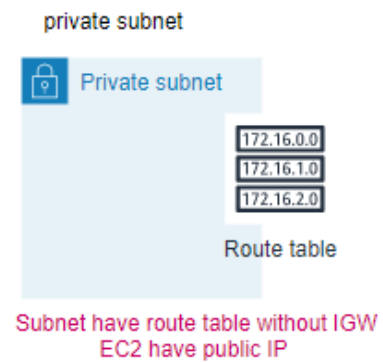
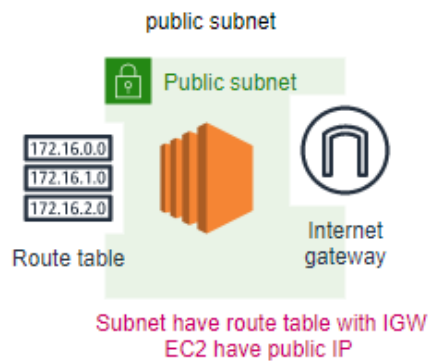
step3: Validate from LB

DNS of ALB

CodeDeploy meant for deploy artifacts on EC2 instance, not to interact with EC2 instance, just like to courier service just focus on their service.

UC: CodeDeploy, Deployment Group for LB.

Public/Private Subnet



NAT (Unidirectional)

